

Propuesta de Arquitectura — SmartLogix

Asignatura: DSY1106 Desarrollo de Software Fullstack III
Integrantes: Benjamin Araneda · Vicente Villalobos
Docente: Víctor Rosendo

1. Introducción

SmartLogix es una plataforma de gestión logística orientada a pequeñas y medianas empresas (PYMEs) del sector eCommerce que operan con sistemas monolíticos. Estos sistemas presentan problemas recurrentes:

- Retrasos en las entregas por falta de trazabilidad en tiempo real.
- Incapacidad para soportar picos de demanda (Cyber Monday, temporada navideña).
- Acoplamiento total entre módulos: un fallo en inventario afecta a pedidos y despacho.
- Dificultad para integrar nuevos canales de venta o servicios externos.

La solución propuesta reemplaza la arquitectura monolítica por un sistema de **microservicios desacoplados**, comunicados mediante eventos, con una capa de seguridad centralizada y un frontend adaptable por rol.

2. Objetivos

#	Objetivo	Cómo lo resuelve la arquitectura
1	Automatizar la gestión logística	Microservicio de Orden con estados automáticos y notificaciones por Kafka
2	Reducir errores en pedidos e inventarios	Validación de stock en tiempo real vía ProductoClient con Circuit Breaker
3	Mejorar tiempos de entrega	Módulo de Envíos con asignación de rutas a transportistas y tracking de estado
4	Escalabilidad horizontal	Cada microservicio corre en su propio contenedor Docker, escalable de forma independiente
5	Integración con otros sistemas	API Gateway como punto único de entrada; eventos Kafka para integración asíncrona

3. Arquitecturas Implementadas

3.1 Microservicios (Spring Boot 4.0.5)

Cada dominio de negocio es un servicio independiente con su propia base de datos PostgreSQL. La comunicación síncrona entre servicios se hace mediante REST (RestTemplate con Circuit Breaker); la comunicación asíncrona, mediante Apache Kafka.

Ventaja clave: si el microservicio de Usuarios cae, el microservicio de Orden sigue funcionando (Circuit Breaker devuelve fallback `null` y usa el nombre del request).

3.2 Arquitectura Orientada a Eventos (Apache Kafka)

Al crearse una orden o cambiar su estado, el microservicio `Orden` publica un evento en Kafka. El microservicio `Inventario` consume ese evento y descuenta el stock de forma asíncrona, sin necesidad de una llamada REST directa.

```
Orden crea pedido → Kafka (orden-creada-topic) → Inventario descuenta stock
Orden cambia estado → Kafka (estado-orden-topic) → otros servicios interesados
```

Si el microservicio `Inventario` está caído, el mensaje queda en la cola de Kafka hasta que se recupere, garantizando consistencia eventual sin pérdida de datos.

3.3 Contenedores Docker (Docker Compose)

Todos los microservicios, la base de datos PostgreSQL, Apache Kafka y Zookeeper están definidos en un archivo `docker-compose.yml`. Esto permite:

- Levantar todo el sistema con un solo comando: `docker compose up --build`
- Versionar la infraestructura junto al código fuente
- Escalar servicios individualmente: `docker compose up --scale orden=3`
- En producción, esta base se migra fácilmente a Kubernetes para auto-scaling real

4. Microservicios Definidos

Microservicio	Puerto	Responsabilidad
Gateway (BFF)	8080	Punto único de entrada. Valida JWT, inyecta <code>userId</code> y <code>rol</code> como headers
Rol	8081	Catálogo de roles del sistema (admin, bodeguero, transportista, cliente)
Users	8082	Registro, autenticación, perfil y dirección del usuario. RUT encriptado con BCrypt
Inventario	8083	Bodegas, pasillos, estantes y stock de productos. Consume eventos de Kafka
Orden	8084	Gestión de pedidos, historial de estados, asignación de transportistas
Producto	8085	Catálogo de productos con stock, precio y país de origen
Estado	8086	Catálogo de estados del ciclo de vida de una orden

Flujo de una orden completa

```
Cliente crea pedido (Angular)
→ Gateway valida JWT
```

```
→ Orden valida stock (ProductoClient)
  → Orden guarda pedido en BD
    → Kafka notifica a Inventario (descuenta stock)
      → Admin/Bodeguero aprueba
        → Transportista toma ruta
          → Transportista marca Entregado
            → Cliente confirma recibo
```

5. API Gateway como BFF (Backend for Frontend)

El Gateway no es un simple proxy: actúa como **Backend for Frontend (BFF)**, adaptando las respuestas del backend a lo que el frontend necesita.

Responsabilidades del Gateway:

1. Recibir todas las peticiones del frontend Angular (puerto 4200 → 8080)
2. Validar el token JWT y rechazar peticiones no autenticadas (401)
3. Extraer **userId** y **rol** del token y agregarlos como headers HTTP internos
4. Enrutar **/api/ordenes/**** → Orden (8084), **/api/usuarios/**** → Users (8082), etc.
5. Liberar a cada microservicio de la lógica de autenticación

```
Angular → GET /api/ordenes/mis-ordenes
Header: Authorization: Bearer eyJ...
  ↓ Gateway extrae userId y rol
→ Orden recibe: X-User-Id: uuid, X-User-Role: cliente
  ↓ Orden filtra solo las órdenes del cliente
← 200 OK [lista de órdenes del cliente]
```

6. Patrones de Diseño Implementados

6.1 Factory Method — **OrdenFactory**

Centraliza la creación de los objetos del dominio, garantizando que siempre se inicialicen con los campos obligatorios correctos.

```
// En lugar de hacer new OrdenModel() y setear campos en el servicio:
OrdenModel orden    = OrdenFactory.crearOrden(userId, nombre, direccionId);
DetalleOrdenModel d = OrdenFactory.crearDetalle(orden, productoId, nombre, precio,
cantidad);
HistorialModel h     = OrdenFactory.crearHistorial(orden, estadoId, estadoNombre,
comentario);
```

Beneficio: Si se agrega un campo obligatorio (ej. **fechaCreacion**, **auditUser**), solo se modifica la factory, sin tocar el servicio.

6.2 Circuit Breaker — Resilience4j

Protege las llamadas REST entre microservicios. Si el servicio destino tarda o falla, el circuit breaker activa el fallback inmediatamente sin esperar un timeout largo.

```
circuitBreakerFactory.create("usersClient").run(
    () -> restTemplate.getForObject(usersUrl + "/api/users/" + userId, Map.class),
    throwable -> { log.error("[CircuitBreaker] ..."); return null; } // fallback
);
```

Estados del Circuit Breaker: CLOSED (normal) → OPEN (servicio caído, fallback activo) → HALF-OPEN (prueba de recuperación).

6.3 Repository Pattern — Spring Data JPA

La capa de servicio nunca escribe SQL directamente; delega en interfaces que extienden `JpaRepository`.

```
public interface OrdenRepository extends JpaRepository<OrdenModel, Long> {
    List<OrdenModel> findById(UUID userId);
    List<OrdenModel> findByTransportistaId(UUID transportistaId);
}
```

Beneficio: Desacopla la lógica de negocio del motor de persistencia. Cambiar de PostgreSQL a otro motor solo requiere cambiar el driver, no el servicio.

6.4 Observer / BehaviorSubject — Frontend Angular

`OrdenService` mantiene un `BehaviorSubject<Orden[]>` que emite la lista actualizada cada vez que se recarga del backend. Todos los componentes suscritos reciben los cambios automáticamente.

6.5 Interceptor HTTP — Frontend Angular

`JwtInterceptor` agrega el header `Authorization: Bearer <token>` a cada petición saliente de forma automática, sin que los componentes deban hacerlo manualmente.

7. Seguridad

Capa	Mecanismo	Detalle
Autenticación	JWT (JSON Web Token)	El usuario hace login en <code>/api/usuarios/login</code> y recibe un token con <code>userId</code> , <code>rol</code> y expiración
Validación	Gateway Filter	Cada petición pasa por <code>JwtGatewayFilter</code> antes de llegar al microservicio
Autorización	Spring Security por rol	Cada endpoint verifica <code>ROLE_admin</code> , <code>ROLE_bodeguero</code> , <code>ROLE_transportista</code> , <code>ROLE_cliente</code>

Capa	Mecanismo	Detalle
Privacidad de datos	BCrypt (RUT)	El RUT del usuario se encripta con BCrypt antes de persistir; no se almacena en texto plano
Privacidad por rol	Filtro en respuesta	La dirección completa y el RUT solo se devuelven al propio usuario o al admin

8. Pipeline CI/CD (GitHub Actions)

El proyecto incluye un pipeline automatizado que se ejecuta en cada push a **develop**:

1. **Compilación** — `mvn compile` con Java 21
2. **Tests unitarios** — `mvn verify` ejecuta JUnit 5 + Mockito (72 tests)
3. **Cobertura** — JaCoCo genera reporte de cobertura de código
4. **Análisis estático** — SonarCloud analiza calidad, bugs y vulnerabilidades
5. **Build Docker** — `mvn package -DskipTests` genera el JAR para la imagen

9. Estrategia de Branching

Se adoptó **Git Flow simplificado** adaptado al trabajo en equipo con múltiples microservicios:

Rama	Propósito
main	Código estable. Solo recibe merges desde develop
develop	Integración. Rama activa de trabajo para el backend
frontend	Desarrollo de la aplicación Angular
feature/bff-gateway	Implementación del API Gateway
feature/inventario-implementacion	Microservicio Inventario
feature/producto-implementacion	Microservicio Producto
feature/tests-unitarios	Tests unitarios con JUnit 5 + Mockito

Convención de commits: `tipo(alcance): descripción` — ej. `feat(orden): agregar asignación de transportista`

10. Requerimientos Funcionales

RF	Descripción	Implementación
RF01	Registro y autenticación de usuarios	Microservicio Users — <code>/registro</code> , <code>/login</code> con JWT
RF02	Control de acceso por rol	Gateway valida JWT + Spring Security por endpoint
RF03	Mantener stock en tiempo real	Microservicio Inventario con bodegas, pasillos y estantes

RF	Descripción	Implementación
RF04	Descontar inventario al crear orden	Evento Kafka <code>orden-creada-topic</code> → Inventario descuenta
RF05	Procesar y validar pedidos	Microservicio Orden con <code>OrdenModel</code> , <code>DetalleOrdenModel</code>
RF06	Gestión automática de estados	<code>HistorialModel</code> como bitácora de cambios de estado
RF07	Coordinar entregas y rutas	Asignación de transportista, tomar/liberar ruta, tracking de estado
RF08	Confirmar recibo por el cliente	Cliente solo puede confirmar una vez que el transportista marcó Entregado

11. Requerimientos No Funcionales

RNF	Descripción	Implementación
RNF01	Escalabilidad horizontal	Cada microservicio en su propio contenedor Docker
RNF02	Rendimiento	Separación de dominios elimina contención; Circuit Breaker evita timeouts largos
RNF03	Disponibilidad	Si un servicio cae, los demás siguen; mensajes en cola Kafka esperan recuperación
RNF04	Consistencia eventual	Eventos asincrónicos via Kafka entre Orden e Inventario
RNF05	Resiliencia	Circuit Breaker (Resilience4j) previene fallos en cascada
RNF06	Seguridad centralizada	API Gateway valida JWT antes de enrutar cualquier petición
RNF07	Mantenibilidad	Repository Pattern + Factory Method + tests unitarios (72 tests)
RNF08	Trazabilidad	Historial de estados por orden; logs estructurados en cada microservicio

12. Conclusión

La arquitectura implementada para SmartLogix cumple con los objetivos planteados:

- **Escalabilidad:** cada microservicio es un contenedor Docker independiente, escalable sin afectar a los demás.
- **Resiliencia:** la combinación de Kafka (eventos) y Resilience4j (Circuit Breaker) garantiza que el sistema siga operando ante fallos parciales.
- **Seguridad:** JWT centralizado en el Gateway, autorización por rol en cada endpoint, y encriptación de datos sensibles (RUT).
- **Calidad del código:** Factory Method, Repository Pattern, 72 tests unitarios y análisis estático con SonarCloud.

- **Experiencia de usuario:** interfaz Angular con roles diferenciados, estados en tiempo real y notificaciones toast.

El sistema no solo resuelve los problemas logísticos de las PYMEs sino que establece una base técnica robusta, mantenible y lista para escalar hacia producción con Kubernetes.

Repositorio: <https://github.com/benjazzx/SmartLogix>

Rama backend: `develop` | **Rama frontend:** `frontend`